



MCDI500 • PROGRAMACIÓN PARA LA CIENCIA DE DATOS
MAGÍSTER EN CIENCIA DE DATOS E INTELIGENCIA ARTIFICIAL
UNIVERSIDAD ANDRÉS BELLO

Análisis de ofertas en licitaciones públicas del sector Salud

Avance Fase 3 – Semana 2: Núcleo algorítmico, eficiencia e implementación orientada a objetos
Repositorio: github.com/vbravo29/mcdi500-licitaciones-salud

Grupo 5

Víctor Bravo Barrera

Nayadeth Garrido Ibáñez

Mauricio Cid

Docente: Omar Salinas Silva

Fecha: 27 de septiembre de 2026

Índice de contenidos

Introducción.....	3
II. Diseño de soluciones algorítmicas eficientes.....	3
A. Codificación funcional y arquitectura básica del script.....	3
B. Preprocesamiento y transformación del dataset.....	4
C. Validación técnica y verificación del código.....	6
D. Eficiencia y optimización.....	7
E. Diseño estructurado del código.....	9
III. Implementación de código modular y robusto.....	10
A. Programación orientada a objetos.....	10
B. Documentación de arquitectura y funcionalidad del código.....	12
IV. Repositorio GitHub (F3).....	13
V. Notebook ejecutable (F3).....	14
Conclusiones y proyección.....	16
VI. Bibliografía.....	16
Anexos.....	18
Anexo A. Matriz de trazabilidad.....	18
Anexo B. Registro de la suite de pruebas.....	19
Anexo C. Comandos de reproducción.....	20

Introducción

Este informe presenta el avance de la Fase 3 del proyecto transversal de análisis de ofertas en licitaciones públicas del sector Salud, construido sobre el reporte de ChileCompra de marzo de 2026 (44.226 filas y 74 columnas). La Fase 1 configuró el entorno reproducible y la Fase 2 implementó el pipeline de obtención, limpieza y transformación. Esta fase integra ese trabajo con un núcleo algorítmico orientado a objetos, compara implementaciones alternativas del cálculo principal y mide su eficiencia temporal y espacial, de acuerdo con los lineamientos de la Evaluación Formativa 3 y de la Evaluación Sumativa 2 del curso (Universidad Andrés Bello, 2026a, 2026b).

El problema analítico se mantiene: describir cómo varía la proporción de ofertas ganadoras según el tamaño del proveedor y el tipo de licitación, en procesos adjudicados y con un denominador propio por grupo. La unidad de observación sigue siendo una oferta asociada a un ítem, no una licitación. El foco de esta fase está en la estructura, la corrección y la eficiencia del código, más que en ampliar el análisis estadístico. El trabajo se concentra en tres decisiones verificables:

- **Núcleo orientado a objetos:** las funciones de `src/proyecto.py` se reorganizaron en un contrato de esquema, un lector abstracto con una implementación CSV, un limpiador con estado encapsulado y un validador compuesto por reglas intercambiables.
- **Cálculo de proporciones:** se compararon cuatro implementaciones equivalentes (referencia de F2, iterativa, recursiva por división y conquista, y agrupada con pandas). Se seleccionó la agrupada, unas 1,16 veces más rápida que la referencia en el conjunto completo.
- **Lectura de datos:** leer solo las siete columnas necesarias, con tipos definidos, hace el flujo de lectura, limpieza y proporciones unas 2,4 veces más rápido y reduce cerca de un 95 % la memoria del DataFrame leído, sin alterar los resultados.

Todas las cifras de este documento provienen de `F3/F3_Algoritmos.ipynb` y de las evidencias guardadas en el repositorio. El Anexo A relaciona cada componente del avance con el archivo que lo respalda.

II. Diseño de soluciones algorítmicas eficientes

Este apartado describe cómo se estructuró el código, cómo se prepararon y validaron los datos y cómo se compararon las implementaciones. El flujo completo sigue cinco pasos: leer, limpiar, validar, calcular y medir. Cada paso recibe una entrada definida y termina con un resultado comprobable antes de avanzar al siguiente.

A. Codificación funcional y arquitectura básica del script

El código se distribuye en módulos de `src/` según su responsabilidad. Los notebooks solo configuran, invocan y presentan resultados; la lógica reside en funciones y clases reutilizables que también ejecutan las pruebas y los scripts de medición. La Tabla 1 resume esta separación.

Tabla 1*Módulos del proyecto y su responsabilidad*

Módulo	Responsabilidad	Elementos principales
src/datos.py	Contrato de esquema, lectura, exportación y huellas SHA-256.	ContratoEsquema, LectorDatos, LectorCSV
src/preprocesamiento.py	Transformaciones puras de fechas, categorías y variables derivadas.	convertir_fechas, normalizar_categorias
src/pipeline.py	Coordinación de la limpieza y registro de su ejecución.	LimpiadorLicitaciones
src/validacion.py	Reglas de calidad y validador que las ejecuta.	ReglaValidacion, seis reglas, ValidadorDatasetProcesado
src/analisis.py	Exploración, frecuencias y cálculo de proporciones.	Cuatro variantes tabla_proporciones_*
src/entorno.py, src/ejecucion.py	Versiones del entorno y ejecución verificada de notebooks.	registro_entorno, ejecutar_notebook
src/proyecto.py	Acceso compatible para F1 y F2 mediante importaciones explícitas.	__all__

Nota. Elaboración propia a partir de la estructura del repositorio.

Las tres variantes nuevas del cálculo comparten la misma firma: reciben un DataFrame, el nombre de la variable de agrupación y el estado administrativo seleccionado, y devuelven la tabla de conteos, denominadores y porcentajes que ya producía F2. También comparten dos funciones auxiliares: `_preparar_conteo`, que filtra sin mutar la entrada y codifica cada resultado (0 ganadora, 1 perdedora, 2 otro), y `_tabla_desde_conteos`, que construye la salida. Así, las diferencias medidas corresponden solo a la estrategia de conteo, y las reglas de negocio no se repiten en cada implementación.

```
def tabla_proporciones_agrupada(df, grupo, estado="Adjudicada"):
    """Obtiene los tres conteos mediante una única agrupación de pandas."""
    # Preparación común: filtro por estado, validación de faltantes y códigos 0/1/2
    grupos, codigos, categorias = _preparar_conteo(df, grupo, estado)
    pares = pd.DataFrame({"grupo": grupos, "resultado": codigos})
    matriz = (pares.groupby(["grupo", "resultado"], observed=True, sort=False)
              .size().unstack(fill_value=0)
              .reindex(index=categorias, columns=[0, 1, 2], fill_value=0))
    conteos = dict(zip(categorias, matriz.to_numpy().tolist()))
    return _tabla_desde_conteos(conteos, categorias) # salida común
```

El flujo de datos es verificable en cada etapa: la lectura comprueba la huella SHA-256 del CSV y el contrato de columnas; la limpieza registra filas y columnas de entrada y salida; la validación informa las reglas superadas; y el cálculo se contrasta con la referencia de F2 antes de interpretar resultados o medir tiempos.

B. Preprocesamiento y transformación del dataset

F3 reutiliza el pipeline de F2 sin modificar sus reglas, ahora coordinado por `LimpiadorLicitaciones`. El CSV original se conserva intacto y se verifica por SHA-256 antes de procesarlo. Las 44.226 filas se conservan en todas las etapas. La Tabla 2 resume cada transformación y su justificación; el detalle de las alternativas descartadas está en `docs/DECISIONES_TECNICAS.md` (decisiones D-09 a D-17).

Tabla 2*Transformaciones del pipeline y justificación técnica*

Etapas	Tratamiento aplicado	Justificación
Columnas vacías	Exclusión nominal de LicitacionBaseTipo, ContratoRenovable y UnidadTiempoRenovacion (100 % vacías), con comprobación previa.	No contienen información; excluirlas por nombre evita eliminar columnas que queden vacías por error en otra versión del archivo.
Fechas	Conversión de diez columnas a datetime64; 7.613 valores del año 1900 en FechaEstimadaEvaluacionOfertas pasan a NaT.	El año 1900 es un valor centinela, no una fecha real. Se conservan las filas y una guardia detiene el proceso si una fecha válida queda nula.
Texto	Eliminación de espacios en los extremos de siete categorías.	Unifica etiquetas sin alterar su significado ni los códigos oficiales.
Categoría residual	NoClasificado (3.410 ofertas) se mantiene como quinta categoría de tamaño.	Imputarla por la moda asignaría artificialmente esas ofertas a otro estrato empresarial.
NA parciales	22 columnas con faltantes parciales se cuantifican y documentan, sin imputación.	Ninguna participa en el cálculo principal; imputar introduciría valores no observados.
Variables derivadas	oferta_ganadora, licitacion_adjudicada y plazo_cierre_dias.	Distinguen ofertas perdedoras de ofertas en procesos desiertos o revocados y permiten validar el denominador.
Escalamiento	No se aplica.	La fase compara frecuencias y proporciones; no se entrena un modelo que requiera una escala común, y el plazo se interpreta mejor en días.

Nota. Cifras obtenidas del notebook F2 y de data/DICCIONARIO_VARIABLES.md.

También se revisaron los valores atípicos de plazo_cierre_dias. La variable tiene mediana de 12,0 días, media de 14,4 y máximo de 61,0. Con el criterio de Tukey ($Q3 + 1,5 \cdot RIC = 34,8$ días), 373 ofertas superan el umbral, pero pertenecen a solo 20 licitaciones y el 64 % corresponde a licitaciones públicas mayores a 5.000 UTM (LR), cuya mediana es de 30,4 días frente a 10,8 en las de 100 a 1.000 UTM (LE). Estos plazos reflejan procesos de mayor monto y no errores de registro, por lo que se conservan.

En F3 se agregó una mejora de lectura que no cambia estas reglas: el análisis solo lee las siete columnas que exigen el limpiador y el cálculo, con tipo category en las cuatro categóricas (The pandas development team, s. f.-b, s. f.-c). La limpieza entrega entonces 44.226 filas y diez columnas (las siete leídas más las tres derivadas), y las proporciones resultan idénticas a las de la lectura completa (sección II.D). F1 y F2 conservan la lectura de las 74 columnas porque exploran todo el archivo.

La validación del dataset resultante la realiza ValidadorDatasetProcesado con seis reglas: columnas vacías descartadas, integridad del número de filas, columnas obligatorias presentes, valores admitidos en ResultadoOferta, coherencia de fechas y consistencia de las variables derivadas. Las seis se superan sobre el conjunto completo. Además, F3/verificar_compatibilidad.py volvió a ejecutar F1 y F2 en copias temporales y comprobó que las tablas de proporciones y los dos CSV exportados coinciden

por SHA-256 con los de la primera entrega.

C. Validación técnica y verificación del código

La verificación combina pruebas automatizadas con unittest (Python Software Foundation, s. f.-d) y casos visibles dentro del notebook. La suite contiene 29 pruebas: ocho de algoritmos, catorce del núcleo orientado a objetos y siete de lectura. Todas se ejecutan también en la sección 5 del notebook, cuya salida conserva el registro detallado (Anexo B). La Tabla 3 agrupa los casos cubiertos.

Tabla 3

Cobertura de la verificación

Tipo de caso	Qué se comprueba	Evidencia
Normal	Oráculo manual: grupo A con 2 de 3 ofertas válidas (66,67 %) y total $3/5 = 60\%$; lectura, limpieza y validación del dataset real.	test_oraculo_manual_y_totales; sección 3 del notebook
Equivalencia	Las cuatro implementaciones producen la misma tabla en ambas variables; particiones recursivas con bloques de 1, 3 y 256; muestras aleatorias con semilla.	test_dataset_real_ambas_agrupaciones; secciones 4.3 y 6.1
Límite	Grupo sin resultados válidos (porcentaje NaN, no 0 %); estado sin registros; índices repetidos; resultados desconocidos.	Secciones 5.2 y 5.3; test_faltantes_resultados_e_indices_duplicados
Excepción	Columnas ausentes, faltantes en la variable de grupo, categoría reservada All, bloque recursivo inválido, contrato mal definido, lista de reglas vacía.	Sección 5.4; test_columnas_ausentes, test_bloques_invalidos
Encapsulamiento	Modificar la lista entregada al contrato o el resumen del limpiador no altera el estado interno.	test_resumen_del_limpiador_es_una_copia_defensiva
Lectura	Selección de columnas, tipos definidos, límite de filas y equivalencia de las tres opciones con el dataset real.	test_dataset_real_opciones_equivalentes

Nota. Las 29 pruebas finalizaron correctamente en la ejecución registrada en evidencias/F3_algoritmos_ejecucion.json.

El caso límite más relevante para la interpretación es el grupo sin resultados válidos: el denominador es cero y la función devuelve un porcentaje no definido en lugar de 0 %, porque mostrar 0 % sugeriría una proporción calculable que no existe. Del mismo modo, un faltante en la variable de agrupación detiene el cálculo con un mensaje explícito, para evitar perder categorías de forma silenciosa:

```
entrada_invalida = mini.copy()
entrada_invalida.loc[0, "Grupo"] = None
tabla_proporciones_agrupada(entrada_invalida, "Grupo")
# ValueError: Grupo: resolver faltantes antes de agrupar.
```

La trazabilidad se mantiene en tres niveles: las pruebas en F3/test_*.py, las salidas del notebook ejecutado en un kernel nuevo y los registros JSON de evidencias/, que guardan la fecha, la versión de Python, el número de celdas ejecutadas, los errores y la huella del notebook.

D. Eficiencia y optimización

Diseño experimental. Se midieron las cuatro implementaciones sobre muestras anidadas de 100, 1.000, 10.000 y 44.226 filas, obtenidas de una permutación con semilla 2026. Se usó timeit con siete rondas de tres ejecuciones, calentamiento previo y orden de algoritmos alternado; timeit desactiva la recolección de basura durante cada ronda (Python Software Foundation, s. f.-a). Se informan mínimo, mediana y cuartiles. El tiempo incluye filtrado, preparación, conteo y construcción de la salida, y excluye lectura y limpieza, que se midieron por separado. La memoria se midió en ejecuciones independientes con tracemalloc, cuyo pico corresponde a las asignaciones rastreadas por Python y no a la memoria total del proceso (Python Software Foundation, s. f.-b). Cada medición guarda las observaciones individuales, las versiones y las huellas del código y del dataset; el notebook rechaza mediciones que no correspondan al código actual.

Tabla 4

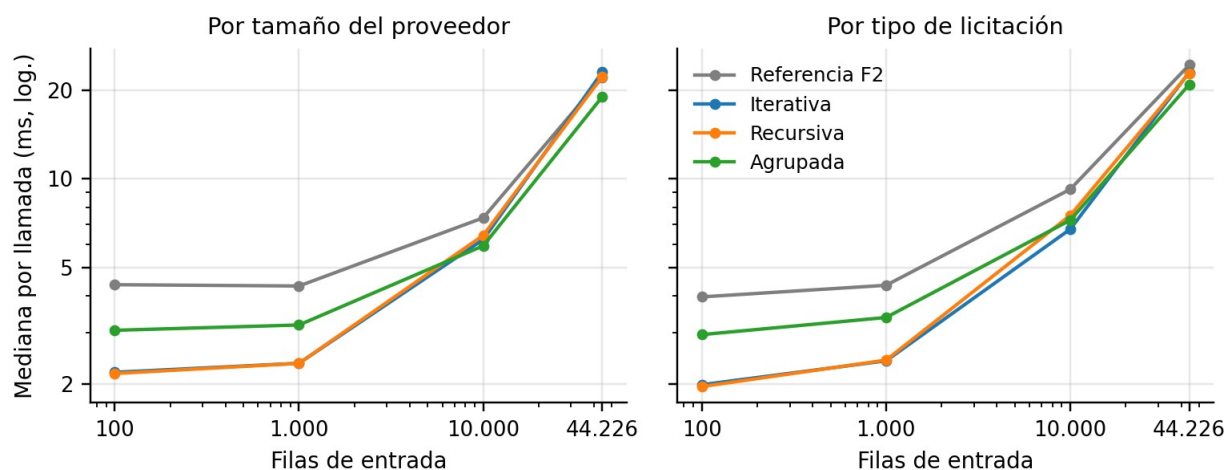
Mediana de tiempo por llamada en el conjunto completo (44.226 filas)

Implementación	Tamaño del proveedor (ms)	Tipo de licitación (ms)	Pico trazado (MiB)
Referencia F2	22,12	24,44	6,51
Iterativa	23,08	23,06	4,55
Recursiva (bloque base 256)	22,28	22,92	4,55
Agrupada (seleccionada)	19,02	20,96	4,55

Nota. Mediana de 7 rondas × 3 ejecuciones; pico: mediana de tres ejecuciones con tracemalloc. Fuente: evidencias/F3_algoritmos/resumen.csv, medición del 27 de septiembre de 2026 en Windows 11 con Python 3.12.10.

Figura 1

Crecimiento de la mediana de tiempo según el número de filas



Nota. Ambos ejes en escala logarítmica. Elaboración propia; gráfico generado en la sección 8.6 del notebook F3 a partir de evidencias/F3_algoritmos/mediciones.json.

Interpretación. Los resultados no muestran una implementación dominante en todos los tamaños. En muestras de 100 y 1.000 filas, la iterativa y la recursiva son las más rápidas ($\approx 2,2$ ms frente a $\approx 3,1$ ms

de la agrupada), porque construir la agrupación de pandas tiene un costo fijo que el recorrido con diccionario no paga. Con 100 filas la recursiva ni siquiera divide, ya que el bloque base es de 256, así que su parecido con la iterativa no es evidencia a favor de la recursividad. Desde 10.000 filas pesa más el recorrido registro a registro en Python y la agrupada, que delega el conteo en código vectorizado mediante groupby (The pandas development team, s. f.-a), pasa a ser la más rápida en el conjunto completo: entre un 9 % y un 18 % menos de tiempo que la iterativa y la recursiva, y unas 1,16 veces más rápida que la referencia F2 en ambas variables.

Un hallazgo relevante es que, con la lectura optimizada, la referencia F2 queda a la par de la iterativa y la recursiva (incluso levemente más rápida por tamaño de proveedor). En la medición anterior, con las 74 columnas, la referencia tardaba 66,9 ms y la agrupada 38,6 ms (razón 1,74). Gran parte del costo de F2 provenía de filtrar y copiar columnas que el cálculo no usa. La optimización más efectiva no fue cambiar el algoritmo de conteo, sino reducir los datos que recibe.

Complejidad. Sea N el total de filas recibidas, n las seleccionadas por estado, g los grupos y b el bloque base, con un número fijo de columnas. El filtro cuesta $O(N)$ y ordenar las categorías $O(g \log g)$. La Tabla 5 resume el costo del conteo, analizado con el esquema de división y conquista de Cormen et al. (2022).

Tabla 5

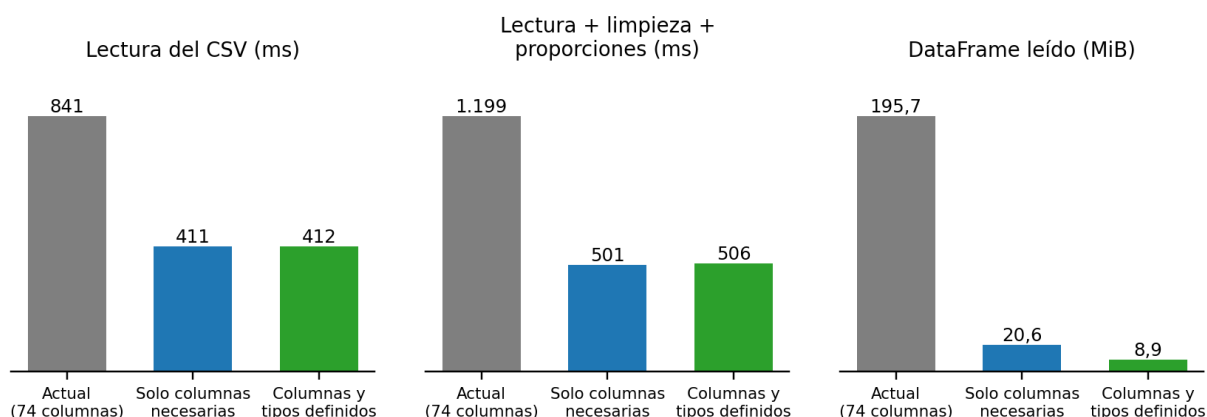
Complejidad temporal y espacial del conteo

Implementación	Tiempo del conteo	Memoria auxiliar del conteo
Iterativa	$O(n)$	$O(g)$
Recursiva	$O(n + S)$; $S \leq \min(g \cdot L, n \cdot \log L)$	Diccionarios parciales y pila de profundidad $O(\log L)$
Agrupada	$O(n + g)$, con tres resultados posibles	$O(n + g)$
Referencia F2	Varios recorridos lineales y agrupaciones	$O(n + g)$, más copias filtradas

Nota. $L = \max(1, n/b \text{ redondeado hacia arriba})$ es el número de hojas y S la suma de entradas combinadas en los nodos internos. Todas las funciones completas materializan la máscara, los datos filtrados y la salida, por lo que la memoria auxiliar total es $O(N + n + g)$.

Con g y b fijos, las tres alternativas nuevas son lineales; las diferencias medidas son de constantes, no de orden. La implementación recursiva es correcta y termina: divide el intervalo por el punto medio, resuelve directamente los bloques de hasta 256 filas y combina los conteos parciales. Sin embargo, conserva los diccionarios de la mitad ya resuelta mientras calcula la otra, por lo que su memoria no se reduce a la pila, y no ofrece una ventaja sostenida de tiempo. Los picos trazados son prácticamente iguales entre las tres alternativas nuevas (4,55 MiB) y menores que los de la referencia (6,51 MiB), lo que indica que domina la preparación compartida.

Optimización de la lectura. El archivo tiene 74 columnas y el análisis utiliza siete, por lo que se evaluó si seleccionar columnas (usecols), declarar tipos al leer y usar variables categóricas reducía el costo de carga. Se compararon tres configuraciones de LectorCSV. Antes de medir se comprobó que las tres producen los mismos datos limpios, la misma validación y las mismas proporciones. La Figura 2 resume los resultados.

Figura 2*Tiempo y memoria de las tres formas de leer el CSV*

Nota. Medianas de siete rondas sobre el archivo completo. Elaboración propia; gráfico generado en la sección 12.11 del notebook F3 a partir de F3/mediciones_lectura/mediciones.json.

Leer 7 de 74 columnas reduce el tiempo de lectura de 841 ms a 411 ms. La reducción es a la mitad y no a una décima parte, porque `usecols` evita crear las columnas descartadas, pero el lector debe igualmente recorrer cada línea completa (The pandas development team, s. f.-b). La lectura crece de forma aproximadamente proporcional a $N \cdot C$, y la ventaja aumenta con el volumen: la razón entre la lectura completa y la selectiva pasa de 1,59 con 1.000 filas a 2,05 con el archivo completo. Definir tipos `category` no cambia el tiempo (412 ms), pero reduce el DataFrame leído de 20,6 a 8,9 MiB, porque cada categoría se almacena una vez y las filas guardan códigos enteros (The pandas development team, s. f.-c). En el flujo completo de lectura, limpieza y proporciones, el tiempo baja de 1.199 ms a 506 ms (2,4 veces) y el DataFrame limpio de 171,5 a 17,6 MiB.

Elección final. Para el análisis de F3 se adopta la combinación de lectura con columnas y tipos definidos y cálculo con la variante agrupada. La elección se justifica en términos de tiempo, porque es la opción más rápida en el conjunto completo para ambas variables; de espacio, porque tiene el menor pico de memoria junto con las otras alternativas nuevas y usa un DataFrame 95 % más pequeño; y de corrección, porque todas las alternativas se verificaron contra la referencia. La conclusión se limita al volumen, la cardinalidad, el equipo y las versiones registradas: los datos no justifican afirmar que pandas sea siempre más rápido, y en tablas pequeñas el recorrido iterativo sería preferible. La función de F2 se conserva sin cambios por compatibilidad.

E. Diseño estructurado del código

La arquitectura se organiza por capas con dependencias en un solo sentido: la lectura no conoce la limpieza, la limpieza no conoce la validación y el análisis solo recibe un DataFrame ya validado. Cada componente tiene una responsabilidad acotada (alta cohesión) y se comunica con los demás mediante DataFrames y contratos explícitos, no mediante estado compartido (bajo acoplamiento). La Tabla 6 muestra el flujo y la comprobación que cierra cada paso.

Tabla 6*Flujo de datos y comprobación antes de avanzar*

Paso	Componente	Salida verificable
1. Leer	LectorCSV + ContratoEsquema	Huella SHA-256, columnas requeridas y registros no vacíos.
2. Limpiar	LimpiadorLicitaciones	44.226 filas conservadas; 7 → 10 columnas; resumen de ejecución.
3. Validar	ValidadorDatasetProcesado	Seis reglas superadas.
4. Calcular	tabla_proporciones_agrupada	Equivalencia con la referencia F2 en ambas variables.
5. Medir	F3/medir_algoritmos.py, F3/medir_lectura.py	Tiempos, memoria, versiones y huellas en evidencias/.

La división funcional también se aplicó a los scripts de medición: medir, medir_caso, medir_tiempos, medir_memoria, resumir_medicion y ejecutar tienen una tarea cada uno, y medir_lectura.py reutiliza las mismas funciones para que ambos experimentos sigan la misma metodología. La estructura es extensible: una nueva fuente de datos requiere una subclase de LectorDatos, una nueva validación una subclase de ReglaValidacion y una nueva estrategia de conteo una función con la misma firma, sin modificar los componentes existentes.

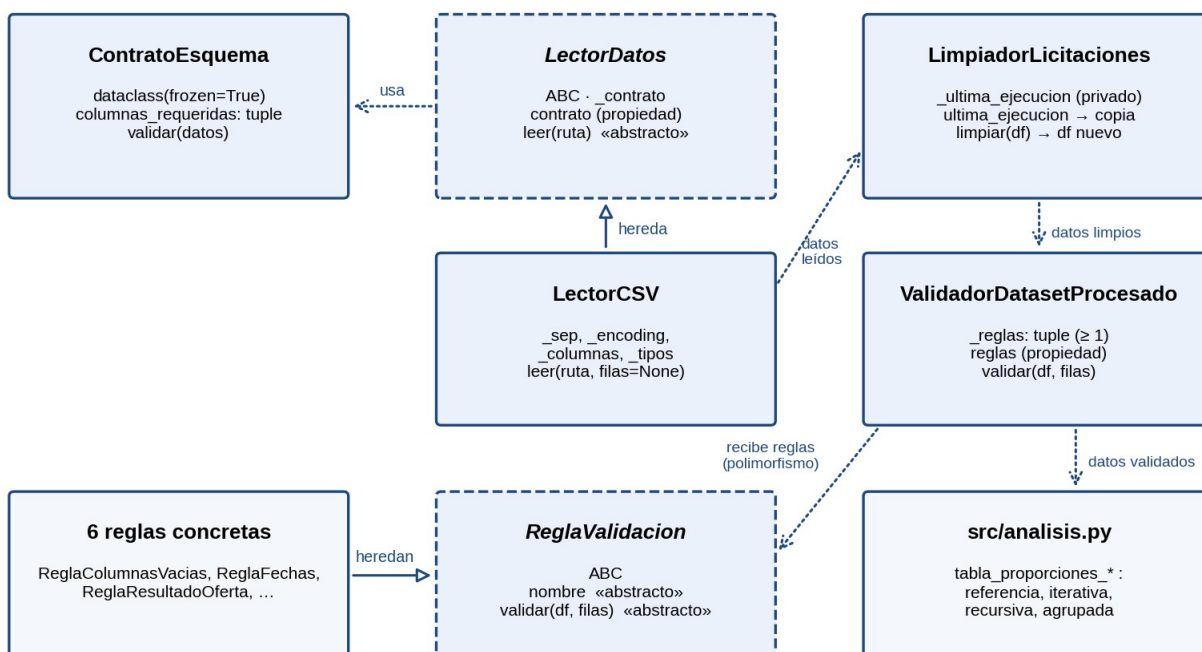
III. Implementación de código modular y robusto

A. Programación orientada a objetos

Las quince funciones de src/proyecto.py desarrolladas en F2 se reorganizaron en clases según su responsabilidad: un lector con su contrato de esquema, un limpiador con su estado y un validador con sus reglas. Se evitó una clase única que concentrara todo el proceso, siguiendo el principio de responsabilidad única descrito por Lott y Phillips (2021). La Figura 3 presenta las clases y sus relaciones.

Figura 3

Clases del núcleo F3 y sus relaciones



Nota. Las cajas con borde discontinuo son clases abstractas (ABC). Las flechas sólidas indican herencia y las punteadas, uso o flujo de datos. Elaboración propia.

Tabla 7

Clases principales del núcleo

Clase	Responsabilidad	Principio que evidencia
ContratoEsquema	Declara las columnas mínimas y valida presencia de columnas y registros.	Encapsulamiento (inmutable)
LectorDatos	Interfaz abstracta de lectura sujeta a un contrato.	Abstracción y herencia
LectorCSV	Lee el CSV con separador, codificación, columnas y tipos configurables.	Herencia y polimorfismo
LimpiadorLicitaciones	Coordina las transformaciones sin mutar la entrada y registra su ejecución.	Encapsulamiento
ReglaValidacion	Interfaz abstracta de una regla de calidad (nombre, validar).	Abstracción y herencia
Seis reglas concretas	Cada una comprueba un aspecto del dataset procesado.	Herencia
ValidadorDatasetProcesado	Ejecuta una colección de reglas intercambiables.	Polimorfismo y composición

Herencia. LectorDatos y ReglaValidacion heredan de ABC y declaran métodos abstractos, de modo que Python impide instanciar una subclase que no los implemente (Python Software Foundation, s. f.-c).

LectorCSV implementa leer y cada regla implementa nombre y validar.

Polimorfismo. ValidadorDatasetProcesado recorre sus reglas e invoca el mismo método sin conocer qué comprueba cada una. Las pruebas lo demuestran con ReglaSiempreValida, una regla definida únicamente en las pruebas, que el validador ejecuta sin cambios en su código. El siguiente fragmento simplificado muestra el recorrido:

```
def validar(self, df, filas_esperadas):
    # Fragmento: se omiten las validaciones de tipo de la entrada
    superadas = []
    for regla in self._reglas:          # cada regla es una subclase distinta
        regla.validar(df, filas_esperadas) # misma llamada, comportamiento propio
        superadas.append(regla.nombre)
    return {"reglas_superadas": len(superadas), "detalle_reglas": tuple(superadas),
            "estado": "OK"}
```

Encapsulamiento. El estado interno se protege en tres puntos. ContratoEsquema es una dataclass congelada que convierte las columnas recibidas en una tupla propia, de modo que modificar la lista original no altera el contrato. LimpiadorLicitaciones guarda su resumen en el atributo privado `_ultima_ejecucion` y lo expone mediante una propiedad que devuelve una copia. ValidadorDatasetProcesado almacena sus reglas en una tupla y distingue la ausencia de reglas (None, que activa las seis predeterminadas) de una colección vacía, que rechaza. Una prueba específica comprueba esta distinción, de modo que una lista vacía nunca active en silencio las reglas predeterminadas.

```
@property
def ultima_ejecucion(self):
    """Resumen defensivo de la ejecución más reciente, o None."""
    # Se entrega una copia: modificarla no altera la trazabilidad del objeto
    return None if self._ultima_ejecucion is None else self._ultima_ejecucion.copy()
```

Diseño sostenible. Cada método público valida sus argumentos y comunica qué corregir, por ejemplo: «Las columnas a leer omiten requeridas del contrato». Todas las clases tienen docstrings. LimpiadorLicitaciones mantiene dos parámetros booleanos (`excluir_vacias`, `normalizar_texto`), heredados de la interfaz de F2, para no romper los notebooks anteriores. Esa firma agrupa varias decisiones en una sola llamada; separar cada transformación en un paso configurable queda como mejora para F4.

B. Documentación de arquitectura y funcionalidad del código

La documentación se distribuye en capas que se refuerzan entre sí. El notebook explica cada bloque con celdas de texto antes del código y comenta las decisiones en las celdas. `docs/DECISIONES_TECNICAS.md` registra 38 decisiones numeradas (D-01 a D-38), con la alternativa descartada y su justificación. `docs/F3_APORTE_ALGORITMOS.md` sintetiza la unidad de análisis, la complejidad, el método de medición y los límites. Los README describen la estructura y los comandos de ejecución.

En la arquitectura se aplican cuatro patrones de diseño básicos. El primero es **Strategy**: las cuatro implementaciones del cálculo comparten firma y son intercambiables, lo que permitió medirlas en igualdad de condiciones. El segundo, **Template Method** mediante clases abstractas: LectorDatos y ReglaValidacion fijan la interfaz y las subclases aportan el comportamiento. El tercero es la **composición**: el validador se construye con una colección de reglas en lugar de heredar de ellas. El

cuarto es una **fachada de compatibilidad**: `src/proyecto.py` reexporta funciones y clases, de modo que F1 y F2 siguen funcionando mientras cada definición tiene una sola implementación. Estos patrones siguen las prácticas de diseño orientado a objetos en Python descritas por Lott y Phillips (2021) y la organización de flujos con pandas propuesta por McKinney (2022).

La evolución del proyecto queda registrada en el historial de Git (sección IV) y en la bitácora de decisiones, que conserva las superadas. Por ejemplo, D-24 describe la organización por funciones de F2 y queda marcada como superada por D-25 a D-27 en F3. La reproducibilidad del conjunto sigue las recomendaciones de Samuel y Mietchen (2024): dependencias declaradas, ejecución en kernel nuevo, huellas de datos y código, y evidencias versionadas.

IV. Repositorio GitHub (F3)

El repositorio está disponible en <https://github.com/vbravo29/mcdi500-licitaciones-salud>. Conserva las carpetas de F1 y F2 sin cambios en sus resultados y añade la carpeta F3, diferenciada del resto, con el notebook, las pruebas, los scripts de medición y verificación, y los archivos de ejemplo. La Tabla 8 resume su organización.

El repositorio se creó con el nombre `sumativo-1` porque en su origen solo contenía el primer entregable (Fases 1 y 2). Al continuar el proyecto en la Fase 3 se renombró a `mcdi500-licitaciones-salud`, un nombre que identifica el curso y el caso de estudio en lugar de una evaluación puntual y que seguirá siendo válido en F4. El cambio conserva el historial completo de commits y GitHub redirige automáticamente el enlace anterior, por lo que las referencias de la primera entrega siguen funcionando. Las salidas guardadas del notebook F2 que muestran rutas locales con el nombre antiguo se mantienen sin cambios, porque forman parte de la evidencia de ejecución de esa entrega.

Tabla 8

Organización del repositorio

Carpeta o archivo	Contenido
F1/, F2/	Notebooks de definición y preprocesamiento, con sus verificadores.
F3/	F3_Algoritmos.ipynb, test_algoritmos.py, test_nucleo_poo.py, test_lectura.py, medir_algoritmos.py, medir_lectura.py, verificar_algoritmos.py, verificar_compatibilidad.py, fixtures/, mediciones_lectura/.
src/	Módulos de lectura, preprocesamiento, limpieza, validación, análisis y entorno.
data/	CSV original versionado con su huella; processed/ excluido de Git y regenerado por F2, según lo declara el README.
evidencias/	Registros de ejecución de F1, F2 y F3, mediciones de algoritmos y verificación de compatibilidad.
docs/	Informes, mapa conceptual, decisiones técnicas y análisis de algoritmos.
README.md, F3/README.md	Instrucciones de instalación, ejecución del notebook F3, dependencias y organización.
requirements.txt, .mail map	Dependencias de F1 a F3 e identidades consolidadas de los integrantes.

La Tabla 9 muestra los commits que registran la evolución del núcleo algorítmico, desde la

incorporación de las clases hasta la aplicación de la lectura optimizada.

Tabla 9

Evolución de la Fase 3 en el historial de Git

Fecha	Commit	Autor	Cambio
23-09	8087c45	Nayadeth Garrido	Incorporación inicial del núcleo POO: contrato, lectores, limpiador y validador.
24-09	189f251	Víctor Bravo	Separación de funciones por responsabilidad con compatibilidad F1/F2.
26-09	48e62d3	Víctor Bravo	Rekursividad justificada, mediciones y equivalencia de algoritmos.
27-09	d90111e	Nayadeth Garrido	Cierre de limpieza y validación con casos de error.
27-09	7f66d24	Mauricio Cid	Lectura configurable de columnas, tipos y filas en LectorCSV.
27-09	f1c2a0c · 86da721	Mauricio Cid	Script de comparación de lectura y sus pruebas.
27-09	e41c1fd	Mauricio Cid	Integración de la comparación de lectura en el notebook F3.
27-09	d9d8227	Víctor Bravo	Documentación, verificación de compatibilidad y evidencias.
27-09	7a0ead5	Mauricio Cid	Aplicación de la lectura optimizada en el análisis.

Nota. Fechas de 2026. Se omiten los merges y los commits documentales menores.

Contribuciones individuales. Desde la entrega anterior el trabajo se repartió por módulo, y el historial registra aportes de los tres integrantes: Víctor Bravo, en la separación modular, los algoritmos, las mediciones y el notebook F3; Nayadeth Garrido, en el núcleo POO de limpieza y validación, los casos de error y el informe; y Mauricio Cid, en la lectura configurable, la comparación de lectura, sus pruebas y el registro del entorno. El archivo .mailmap consolida los nombres históricos de cada integrante sin reescribir el historial. El autor alexander corresponde a un exintegrante que se retiró del grupo. Los cambios se integran en la rama desarrollo y pasan a main mediante pull request.

V. Notebook ejecutable (F3)

F3/F3_Algoritmos.ipynb concentra el desarrollo técnico de esta fase. Tiene 75 celdas (34 de código) organizadas en 15 secciones, y cada celda de código tiene un objetivo verificable, precedido de su explicación. La Tabla 10 muestra la correspondencia entre las secciones del notebook y este informe.

Tabla 10*Estructura del notebook F3 y correspondencia con el informe*

Secciones	Contenido	Informe
1	Entorno, versiones y funciones de presentación.	V
2	Lectura con contrato, limpieza y validación mediante las clases.	II.B, III.A
3–4	Contrato del cálculo, ejemplo manual, alternativas y descomposición recursiva.	II.A, II.E
5	Suite de 29 pruebas y casos límite visibles.	II.C
6	Equivalencia de algoritmos y proporciones del dataset real.	II.C, V
7–11	Diseño experimental, tiempos (con gráfico en 8.6), memoria, complejidad y elección.	II.D
12	Comparación de tres formas de leer el CSV (con gráfico en 12.11).	II.D
13–15	Conclusión general, reproducibilidad y referencias técnicas.	Conclusiones, VI

El notebook integra los tres paradigmas del curso. La programación estructurada aparece en las funciones de preparación y conteo; la recursiva, en `_contar_dividiendo`; y la orientada a objetos, en las clases de lectura, limpieza y validación. También documenta cómo se incorporan las mejoras desde F2 y desde el avance formativo: la referencia F2 se conserva y se usa como oráculo, y la sección 11 compara las cifras actuales con las de la medición anterior.

Ejecución reproducible. F3/`verificar_algoritmos.py` ejecuta el notebook en un kernel nuevo y registra el resultado. La última ejecución (27 de septiembre de 2026, Python 3.11.15, con las versiones de `numpy` y `pandas` fijadas en `requirements.txt`) completó las 34 celdas de código en orden y sin errores. Las mediciones guardadas se validan por huella antes de mostrarse y pueden regenerarse con `REGENERAR_MEDICIONES = True`. Las proporciones obtenidas con la variante seleccionada coinciden con las de F2 (Tabla 11).

Tabla 11*Proporción de ofertas ganadoras por tamaño del proveedor (procesos adjudicados)*

Tamaño	Ganadoras	Perdedoras	Ofertas válidas	% Ganadora
Grande	12.575	6.255	18.830	66,78
Mediana	4.982	4.001	8.983	55,46
NoClasificado	1.803	1.576	3.379	53,36
Pequeña	5.022	4.577	9.599	52,32
Micro	1.680	1.554	3.234	51,95
Total	26.062	17.963	44.025	59,20

Nota. Cada porcentaje usa como denominador las ofertas con resultado válido de su grupo. Las diferencias son descriptivas y no implican causalidad. Fuente: notebook F3, sección 6.2.

Los resultados sustantivos no cambian respecto de F2: las empresas grandes ganan el 66,78 % de sus

ofertas y las micro el 51,95 %. Por tipo de licitación, las públicas entre 100 y 1.000 UTM (LE) concentran 23.590 ofertas válidas con un 62,26 % de éxito y las mayores a 5.000 UTM (LR) un 51,45 %. Las licitaciones privadas tienen porcentajes altos pero muy pocas ofertas (entre 2 y 72), por lo que no tienen el mismo respaldo que las categorías numerosas.

Conclusiones y proyección

La Fase 3 transformó un conjunto de funciones de F2 en un núcleo orientado a objetos, con responsabilidades separadas, herencia mediante clases abstractas, polimorfismo en la validación y estado encapsulado, sin alterar los resultados de la primera entrega. La comparación de cuatro implementaciones del cálculo mostró que las diferencias entre alternativas lineales son de constantes y dependen del tamaño de la entrada, y que la variante agrupada es la mejor opción para el conjunto completo. La medición de la lectura aportó la mejora más importante: leer solo lo necesario redujo a menos de la mitad el tiempo del flujo y a una vigésima parte la memoria.

Para F4 quedan tres líneas de trabajo. La primera es la agregación por licitación (1.864 procesos únicos), que exige combinar el estado del proceso con los atributos del proveedor sin perder coherencia. La segunda es conservar el tipo `category` durante la limpieza, cuyo beneficio de memoria hoy se pierde al normalizar texto. La tercera es separar las transformaciones del limpiador en pasos configurables.

VI. Bibliografía

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4.^a ed.). MIT Press. <https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/>
- Lott, S. F., & Phillips, D. (2021). *Python object-oriented programming: Build robust and maintainable object-oriented Python applications and libraries* (4.^a ed.). Packt Publishing. <https://www.packtpub.com/en-us/product/python-object-oriented-programming-9781801077262>
- McKinney, W. (2022). *Python for data analysis: Data wrangling with pandas, NumPy, and Jupyter* (3.^a ed.). O'Reilly Media. <https://wesmckinney.com/book/>
- Python Software Foundation. (s. f.-a). *timeit — Measure execution time of small code snippets*. Python 3 documentation. <https://docs.python.org/3/library/timeit.html>
- Python Software Foundation. (s. f.-b). *tracemalloc — Trace memory allocations*. Python 3 documentation. <https://docs.python.org/3/library/tracemalloc.html>
- Python Software Foundation. (s. f.-c). *abc — Abstract base classes*. Python 3 documentation. <https://docs.python.org/3/library/abc.html>
- Python Software Foundation. (s. f.-d). *unittest — Unit testing framework*. Python 3 documentation. <https://docs.python.org/3/library/unittest.html>
- Samuel, S., & Mietchen, D. (2024). Computational reproducibility of Jupyter notebooks from biomedical publications. *GigaScience*, 13, Artículo giad113. <https://doi.org/10.1093/gigascience/giad113>
- The pandas development team. (s. f.-a). *pandas.DataFrame.groupby*. pandas documentation.

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.groupby.html>

The pandas development team. (s. f.-b). *pandas.read_csv*. pandas documentation.

https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html

The pandas development team. (s. f.-c). *Categorical data*. pandas documentation.

https://pandas.pydata.org/docs/user_guide/categorical.html

Universidad Andrés Bello. (2026a). *Evaluación Formativa 3: Avance del proyecto Fase 3 – Notebook con scripts y mediciones de complejidad* [Instrucciones y rúbrica del curso MCDI500]. Canvas UNAB.

Universidad Andrés Bello. (2026b). *Evaluación Sumativa 2: Avance del proyecto Fase 3 – Núcleo algorítmico, eficiencia e implementación orientada a objetos* [Pauta y rúbrica del curso MCDI500]. Canvas UNAB.

Anexos

Anexo A. Matriz de trazabilidad

Tabla 12

Relación entre los componentes del avance y la evidencia

Componente	Archivo principal	Evidencia	Informe
Codificación y arquitectura	src/*.py	Módulos por responsabilidad y funciones con firma común.	II.A
Preprocesamiento	src/pipeline.py, src/preprocesamiento.py	44.226 filas; seis reglas superadas; compatibilidad por SHA-256.	II.B
Validación técnica	F3/test_*.py	29 pruebas aprobadas; casos límite en la sección 5 del notebook.	II.C, Anexo B
Eficiencia	F3/medir_algoritmos.py, F3/medir_lectura.py	timeit, tracemalloc, datos crudos, versiones y huellas.	II.D
Diseño estructurado	src/, F3/	Flujo de cinco pasos y recursividad por división.	II.E
POO	src/datos.py, src/pipeline.py, src/validacion.py	Herencia, polimorfismo y encapsulamiento con pruebas.	III.A
Documentación	docs/ DECISIONES_TECNICAS. md	38 decisiones con alternativas y justificación.	III.B
Repositorio F3	README.md, F3/README.md, .mailmap	Carpeta F3, instrucciones e historial.	IV
Notebook ejecutado	F3/F3_Algoritmos.ipynb	34 celdas sin errores en kernel nuevo; figuras 1 y 2 en las secciones 8.6 y 12.11.	V

Anexo B. Registro de la suite de pruebas

Resultado de la suite ejecutada en la sección 5.1 del notebook: 29 pruebas, todas aprobadas.

Tabla 13

Registro de la suite de pruebas

Módulo	Prueba	Resultado
test_algoritmos	test_bloques_invalidos	ok
test_algoritmos	test_columnas_ausentes	ok
test_algoritmos	test_dataset_real_ambas_agrupaciones	ok
test_algoritmos	test_faltantes_resultados_e_indices_duplicados	ok
test_algoritmos	test_grupo_faltante_y_categoria_reservada	ok
test_algoritmos	test_oraculo_manual_y_totales	ok
test_algoritmos	test_particiones_aleatorias_reproducibles	ok
test_algoritmos	test_vacio_filtro_sin_filas_y_otro_estado	ok
test_nucleo_poo	test_contrato_conserva_columnas_ante_mutacion_externa	ok
test_nucleo_poo	test_contrato_rechaza_nombres_invalidos	ok
test_nucleo_poo	test_contrato_rechaza_definiciones_invalidas	ok
test_nucleo_poo	test_lector_rechaza_un_contrato_de_otro_tipo	ok
test_nucleo_poo	test_lector_aplica_contrato_y_adaptador_conserva_resultado	ok
test_nucleo_poo	test_lector_rechaza_esquema_incompleto	ok
test_nucleo_poo	test_limpiador_encapsula_estado_y_no_muta_entrada	ok
test_nucleo_poo	test_resumen_del_limpiador_es_una_copia_defensiva	ok
test_nucleo_poo	test_limpiador_rechaza_configuracion_y_entradas_invalidas	ok
test_nucleo_poo	test_validador_por_reglas_y_adaptador	ok
test_nucleo_poo	test_validador_acepta_reglas_polimorficas	ok
test_nucleo_poo	test_validador_distingue_ausencia_de_reglas_de_lista_vacia	ok
test_nucleo_poo	test_validador_rechaza_reglas_y_datos_invalidos	ok
test_nucleo_poo	test_validador_detecta_caso_limite	ok
test_lectura	test_lectura_por_defecto_conserva_f1	ok
test_lectura	test_columnas_seleccionadas_en_orden_del_archivo	ok
test_lectura	test_tipos_definidos_al_leer	ok
test_lectura	test_columna_inexistente_informa_contrato	ok
test_lectura	test_columnas_deben_incluir_contrato	ok
test_lectura	test_filas_limita_la_lectura	ok
test_lectura	test_dataset_real_opciones_equivalentes	ok

Nota. Ran 29 tests ... OK. Salida completa en la sección 5.1 de F3/F3_Algoritmos.ipynb.

Anexo C. Comandos de reproducción

Desde la raíz del repositorio, en Windows con Python 3.12:

```
py -3.12 -m venv .venv
.\venv\Scripts\python.exe -m pip install -r requirements.txt
.\venv\Scripts\python.exe -m unittest F3.test_algoritmos F3.test_nucleo_poo F3.test_lectura -v
.\venv\Scripts\python.exe F3\medir_algoritmos.py
.\venv\Scripts\python.exe F3\medir_lectura.py
.\venv\Scripts\python.exe F3\verificar_algoritmos.py
.\venv\Scripts\python.exe F3\verificar_compatibilidad.py
```